

`</>` beginner's handbook

JavaScript from **Zero**

A hands-on, example-driven guide to learning JavaScript from scratch — variables to your first script in the browser.

ES2020+

no experience needed

runs in any browser

CONTENTS **What's Inside**

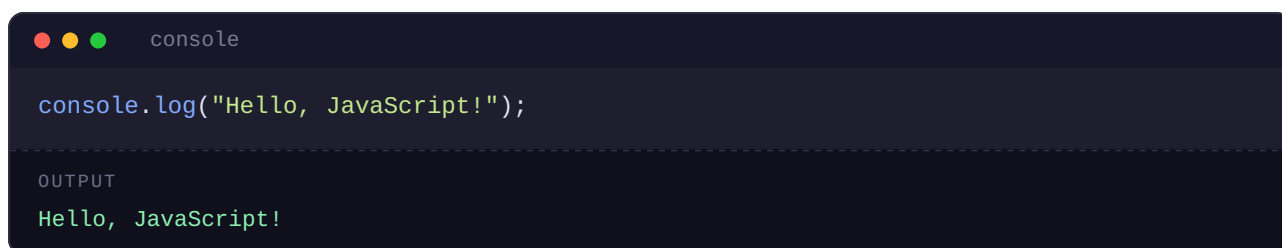
1	What is JavaScript & how to run it	§1
2	Variables — let, const, var	§2
3	Data types	§3
4	Operators & conditionals	§4
5	Loops	§5
6	Functions	§6
7	Arrays & array methods	§7
8	Objects	§8
9	The DOM & events	§9
10	A tiny real project	§10
11	Where to go next	§11

SECTION 1 What Is JavaScript?

JavaScript is the programming language that makes websites interactive — every button click, dropdown, form validation, and live-updating page uses it. It runs in every web browser and, thanks to Node.js, on servers too.

The fastest way to try it: the browser console

Open any web page, right-click → **Inspect** → click the **Console** tab. Type code and hit Enter — it runs instantly. No installation needed.

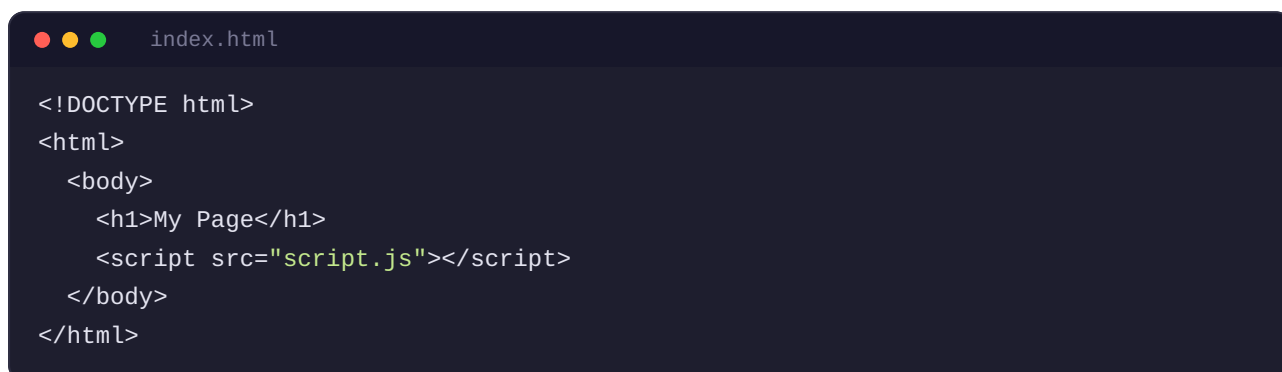
A screenshot of a browser's developer console. The title bar says 'console'. The input area contains the code `console.log("Hello, JavaScript!");`. Below a dashed line, the output shows `Hello, JavaScript!` in green text.

```
console.log("Hello, JavaScript!");
```

OUTPUT

Hello, JavaScript!

Running a script in an HTML page

A screenshot of a code editor showing the content of a file named 'index.html'. The code is a basic HTML document structure with a script tag.

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My Page</h1>
    <script src="script.js"></script>
  </body>
</html>
```

Tip — Save that as `index.html`, create a `script.js` next to it, and double-click the HTML file to open it in your browser. That's your whole setup.

SECTION 2 Variables

Variables store data so you can reuse and change it. JavaScript has three ways to declare one:

```
variables.js

let age = 27;           // can change later
const name = "Tim";     // cannot be reassigned
var oldStyle = "avoid"; // legacy, avoid in new code

age = 28;               // works, let is assignable
// name = "Sam";       // error - const can't be reassigned
```

Keyword	Reassignable?	When to use
<code>let</code>	Yes	Default choice for values that change
<code>const</code>	No	Default choice — use unless you need reassignment
<code>var</code>	Yes	Legacy only — skip it in new code

Rule of thumb — reach for `const` by default, switch to `let` only when the value truly needs to change.

SECTION 3 Data Types

```
types.js

const aString = "hello";    // text
const aNumber = 42;         // integers & decimals, one type
const aBoolean = true;     // true / false
const aList = [1, 2, 3];   // array
const aObject = { city: "Phnom Penh" }; // object
const nothing = null;      // intentionally empty
let notSet;                // undefined until assigned

console.log(typeof aNumber); // "number"
```

JavaScript is **dynamically typed** — you don't declare a type up front, and a variable can hold any type. Use `typeof` to check what something currently is.

SECTION 4 Operators & Conditionals

operators.js

```
10 + 5;   10 - 5;   10 * 5;   10 / 5;   10 % 3; // remainder → 1

5 === 5;   // true  - strict equals (use this one)
5 == "5";  // true  - loose equals, avoid it
5 !== 6;   // true  - not equal
```

if / else

conditionals.js

```
const hour = 14;

if (hour < 12) {
  console.log("Good morning");
} else if (hour < 18) {
  console.log("Good afternoon");
} else {
  console.log("Good evening");
}
```

OUTPUT

Good afternoon

SECTION 5 Loops

loops.js

```
// for – when you know how many times
for (let i = 1; i <= 3; i++) {
  console.log("Lap " + i);
}

// while – when you don't know how many times
let stock = 3;
while (stock > 0) {
  stock--;
}

// for...of – the easiest way to loop over an array
for (const fruit of ["apple", "pear"]) {
  console.log(fruit);
}
```

OUTPUT

```
Lap 1
Lap 2
Lap 3
apple
pear
```

SECTION 6 Functions

A function is a reusable block of code. There are three common ways to write one:

```
functions.js

// classic function declaration
function greet(name) {
  return "Hi, " + name;
}

// arrow function – shorter, very common today
const add = (a, b) => a + b;

// arrow function with a default parameter
const shout = (msg = "hello") => msg.toUpperCase();

console.log(greet("Tim")); // "Hi, Tim"
console.log(add(2, 3));    // 5
console.log(shout());      // "HELLO"
```

Tip — Arrow functions (`=>`) are the modern default for short functions. Reach for a classic `function` when you need it hoisted (usable before its definition) or a method on an object.

SECTION 7 Arrays & Array Methods

```
arrays.js

const nums = [4, 8, 15, 16];

nums.push(23);           // add to the end
nums.length;             // 5

const doubled = nums.map(n => n * 2);
const big      = nums.filter(n => n > 10);
const total    = nums.reduce((sum, n) => sum + n, 0);

console.log(doubled); // [8, 16, 30, 32, 46]
console.log(big);     // [15, 16, 23]
console.log(total);   // 66
```

Method	What it does
<code>.map()</code>	Transforms every item, returns a new array
<code>.filter()</code>	Keeps items that pass a test
<code>.reduce()</code>	Folds the array down to a single value
<code>.find()</code>	Returns the first item that matches
<code>.forEach()</code>	Runs code per item, no new array returned

SECTION 8 Objects

Objects group related data together using **key: value** pairs.

```
objects.js

const user = {
  name: "Tim",
  role: "developer",
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};

console.log(user.name);    // "Tim"
console.log(user.greet()); // "Hi, I'm Tim"

// destructuring - pull values out directly
const { role } = user;
console.log(role);        // "developer"
```

Note — that backtick string (``Hi, I'm ${this.name}``) is a **template literal** — it lets you drop variables straight into a string with `${ }`.

SECTION 9 The DOM & Events

The DOM (Document Object Model) is how JavaScript reads and changes what's on the page.

```
dom.js

const button = document.querySelector("#myBtn");
const title = document.querySelector("h1");

button.addEventListener("click", () => {
  title.textContent = "You clicked it!";
  title.style.color = "#5a5aff";
});
```

`querySelector` finds an element on the page, `addEventListener` reacts to what the user does with it — click, type, hover, submit, and more.

SECTION 10 A Tiny Real Project: Click Counter

Putting it together — variables, a function, and an event listener:

```
counter.html

<button id="counterBtn">Clicked 0 times</button>

<script>
  let count = 0;
  const btn = document.querySelector("#counterBtn");

  btn.addEventListener("click", () => {
    count++;
    btn.textContent = `Clicked ${count} times`;
  });
</script>
```

TRY IT YOURSELF

modify the counter so the button turns red after 5 clicks and resets to 0 after 10.

Hint: you'll need an `if` check inside the click handler.

SECTION 11 Where to Go Next

- **Practice daily** — even 20 minutes a day on small exercises beats one long session a week.
- **Build tiny projects** — to-do list, quiz app, calculator. Small and finished beats big and abandoned.
- **Learn Git** — version control early saves pain later.
- **Read real code** — browse small open-source repos on GitHub once the basics feel comfortable.
- **Then explore** — async/await & promises, ES modules, and a framework (React or Vue) once fundamentals are solid.

You now know enough to build things.

Every JavaScript developer started exactly here — variables, functions, and a `console.log`. The fastest way forward from this point is not more reading, it's writing broken code and fixing it.